



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

SECR2043 - OPERATING SYSTEMS
SEMESTER II, SESSION 2024/2025
SECTION 07

PROJECT REPORT

Synchronization using Semaphore
Printing Queue System

Group : 01

Prepared by : 1. MOHAMAD SAMY ARIDHAN HON BIN M.AMIN HON (A23CS0246)

2. CHEW CHUAN KAI (A23CS0062)

3. LING YU QIAN (A23CS0301)

4. MUHAMMAD FATHURRAHMAN BIN AYUB (A23CS0129)

Lecturer : DR. FARKHANA BINTI MUCHTAR

Date : 06/07/2025

Table of Contents

Abstract.....	3
1.0 Introduction.....	4
2.0 Problem Statement.....	4
3.0 Related Works.....	5
4.0 Methodology.....	6
5.0 Design & Implementation.....	6
6.0 Results & Discussion.....	8
7.0 Future Work & Conclusion.....	10
8.0 Acknowledgement.....	11
References.....	13
Presentation Video.....	13

Abstract

For our SECR2043 Operating Systems project, our team worked on a simulation that shows how semaphores can help manage synchronization in a real-world case where several computers try to use the same printer at the same time. This idea came from the need to control what happens when multiple print requests are made at once, which is common in places like computer labs and offices. This is quite a significant issue as printing jobs can overlap or even fail if there is no proper system in place, which consequently disrupts the work flows of the users.

In this project, we have used Java's built-in multithreading features together with the *Semaphore* class from the *java.util.concurrent* package. Each computer was represented by a thread, trying to print something using one shared printer. We applied binary semaphore as it only allows one thread to enter the critical section at a time. This way, each thread must wait for the printer to be free, or released before its request to print is being carried out. This prevents conflicts and data corruption.

The simulation worked well, and this is indicated with our code's output which clearly shows that only one thread was able to print at a time while the others waited for their turn. This proves that semaphore can manage shared resources effectively as there were no sights of overlapping printing jobs, indicating that the synchronization worked as intended.

During the process of constructing and testing the simulation, we ran into some problems such as threads not interacting properly and deadlocks or delays occurring without clear reasons. But through the mistakes, we have gained experience and understood how semaphore works with greater depth instead of just textbook knowledge. Overall, this project helped us understand the significance of synchronization management in our daily lives and how almost all devices we are using utilize them in order to keep the system stable in multi-user environments.

1.0 Introduction

Various users typically share device resources, such as printers, in contemporary networked environments. In the case of posting multiple print requests simultaneously, common usage could result in serious synchronization problems. These simultaneous access requests can cause systems to crash, data to become corrupted, or concurrent printing processes in case they are poorly synchronized.

Semaphores and other synchronization techniques are used to address this issue. Strong operating system primitives, known as semaphores, assist in controlling access to the shared resources by limiting the processes simultaneously that make use of them. Semaphores are important in sequential task execution and the prevention of race conditions.

An application of a semaphore-based solution to a synchronization issue in a printer queue system is examined within this case study. The primary objective is to prevent contention and ensure data integrity by limiting the number of computers that can access the printer at the same time. Either the counting semaphore (to handle multiple queued tasks) or the binary semaphore (to achieve exclusive usage) can be efficiently utilized based on design.

2.0 Problem Statement

In this digital era, it is very common to see many computers connected to a single printer. Although this is convenient, the problem starts when all of the computers are trying to use the printer at the same moment. If no proper control mechanism is implemented, this type of situation will lead to corrupted output, overlapping printing jobs, or complete failure of printing. These issues arise by the lack of proper synchronization during access handling of the shared printer.

In order to overcome this, a method to make sure that only one computer is allowed to use the printer at any given time needs to be implemented. There exists a concept that perfectly describes this, which is mutual exclusion. It is an important concept in operating systems to keep things in order by avoiding conflicts and in-order interruptions. It helps protect data integrity and allows everything runs smoothly even when multiple users or processes are running concurrently.

In this project, we have utilized semaphores to solve the problem. A semaphore is a control tool that helps manage threads or processes that enter the critical section, which in this case is the printing activity. By using binary semaphore, we are able to make sure that only one thread can access the printer at one time while the others have to wait. When the job is done, the resource is released and the next computer that wants to do the printing can take its turn. This model is created using Java's built-in concurrency features to mirror a real-world printing queue with proper synchronization and handling.

3.0 Related Works

Java implementation of a printing queue system is an excellent practical application demonstrating how semaphores manage access to common resources in multithreading. Semaphores, introduced by Edsger Dijkstra, are a basic synchronization primitive used to address classic problems such as producer-consumer, readers-writers, and dining philosophers that emphasize mutual exclusion and deadlock prevention.

Java has a native support of semaphores through the `java.util.concurrent.Semaphore` class, which provides simple modeling of access controls in multi-threaded applications. Development environments and educational tools like BlueJ, JShell, and Jeliot have also contributed largely to getting students to imagine and experiment with thread synchronization. In addition, educational materials such as *The Little Book of Semaphores* (Downey, 2016) and interactive learning platforms such as VisuAlgo provide informative content about how control through semaphores ensures safe and fair process execution in systems such as printer queues.

4.0 Methodology

Event	Objective	Date
First discussion	Decide on a topic selection, make preliminary plans, and assign tasks to team members.	28/04/2025
Report discussion	Examine and talk about the preliminary report draft to make sure it complies with the project's objectives.	03/06/2025
Second discussion	Refine the report by considering the first discussion's input, note any problems, and make plans for the following steps.	19/06/2025
Report checking	Examine the report carefully to ensure that it is accurate, comprehensive, and compliant with the criteria.	30/06/2025
Third discussion	Discussion and recording the video presentation	02/07/2025
Submission	Submit the final report and video presentation	06/07/2025

5.0 Design & Implementation

The Semaphore class from the `java.util.concurrent` package and Java multithreading are used in the prototype to mimic a printing queue system. It simulates a situation in which a single printer is shared by several computers (represented by threads).

Overview of the Design

Type of Semaphore: Binary (`printerSemaphore = 1`)

The goal is to guarantee mutual exclusion, meaning that only one thread—that is, one print job—can ever access the printer.

Implementation Steps

1) Initialize the Semaphore

```
Semaphore printerSemaphore = new Semaphore(1); // binary semaphore
```

2) Thread Behavior (Each Computer)

```
public void run() {  
  
    try {  
  
        printerSemaphore.acquire(); // Wait (P operation)  
  
        printDocument();           // Critical section  
  
    } finally {  
  
        printerSemaphore.release(); // Signal (V operation)  
  
    }  
  
}
```

3) Concurrency Simulation

Multiple threads are started simultaneously.

Each thread attempts to acquire the semaphore before printing.

The semaphore blocks all but one thread at a time, preventing race conditions.

Efficiency

This simulation effectively illustrates how semaphores control shared resource access, avoid overlapping print jobs, and guarantee that the printer completes each task independently without causing conflicts or corrupting data.

6.0 Results & Discussion

This section shows the output from the printing queue simulation created using Java. It was designed to test the binary semaphore working in a multithreaded setting. The main idea behind running this simulation was to check whether the synchronization process would actually prevent race conditions from happening. At the same time, it is also to ensure that mutual exclusion is maintained when several threads, which represent different computers, try to access the printer at the same time.

```
chuankai@secr2043:~$ nano PrintingQueueSimulation.java
chuankai@secr2043:~$ javac PrintingQueueSimulation.java
chuankai@secr2043:~$ java PrintingQueueSimulation
Computer 1 is trying to print...
Computer 1 has acquired the printer.
Computer 3 is trying to print...
Computer 4 is trying to print...
Computer 2 is trying to print...
Computer 5 is trying to print...
Computer 1 has finished printing.
Computer 1 has released the printer.
Computer 3 has acquired the printer.
Computer 3 has finished printing.
Computer 3 has released the printer.
Computer 4 has acquired the printer.
Computer 4 has finished printing.
Computer 4 has released the printer.
Computer 2 has acquired the printer.
Computer 2 has finished printing.
Computer 2 has released the printer.
Computer 5 has acquired the printer.
Computer 5 has finished printing.
Computer 5 has released the printer.
chuankai@secr2043:~$ |
```

Figure 6.1: Output of the printing queue simulation showing that each computer acquires and releases the printer one at a time, demonstrating successful synchronization using a binary semaphore.

Table 6.1: Line-by-line Breakdown of Simulation Output and Semaphore Behavior

Output Line	Description	Purpose / Synchronization Insight
Computer 1 is trying to print...	Thread for Computer 1 started and attempting to enter critical section	Indicates the initial request to access shared resource (printer)
Computer 1 has acquired the printer.	Thread 1 successfully acquired the binary semaphore	Confirms mutual exclusion: only one thread can enter at a time
Computer 3 is trying to print...	Another thread (Computer 3) starts and requests access	Shows concurrent threads are running in parallel

Computer 4 is trying to print...	Another thread joins, requesting to print	More threads stacking up while semaphore is occupied
Computer 2 is trying to print...	Computer 2 also joins the queue	Multiple pending requests waiting for printer access
Computer 5 is trying to print...	Last thread in the group tries to access the printer	Demonstrates multiple contention for shared resource
Computer 1 has finished printing.	Thread 1 completes its task in critical section	End of job for the first thread, ready to release resource
Computer 1 has released the printer.	Semaphore released by Thread 1	Unlocks the resource, allowing next thread to proceed
Computer 3 has acquired the printer.	Thread 3 acquires the semaphore	Access granted to next waiting thread
Computer 3 has finished printing.	Printing done by Computer 3	Shows orderly processing, no overlaps
Computer 3 has released the printer.	Resource freed again	Makes way for next thread
Computer 4 has acquired the printer.	Thread 4 now enters critical section	Queue follows fair and safe access pattern
Computer 4 has finished printing.	Job completed by Thread 4	Each thread performs its task one at a time
Computer 4 has released the printer.	Resource released	Synchronization continues
Computer 2 has acquired the printer.	Thread 2 granted access	Even though it tried earlier, it waited properly
Computer 2 has finished printing.	Thread completes job	Order of execution not strictly by start time, but safe
Computer 2 has released the printer.	Semaphore freed	Final thread can now proceed
Computer 5 has acquired the printer.	Last thread enters the critical section	All threads successfully served in isolated access

Computer 5 has finished printing.	Task completed safely	No conflicts throughout execution
Computer 5 has released the printer.	Final semaphore release	Ends the simulation in a controlled manner

The output we have retrieved from the simulation shows that binary semaphore works well when it comes to managing and controlling several computers that are trying to use a shared printer. In the terminal, there are five different computers, each running as a separate thread and trying to print their own document. From observation, the key points that show synchronization are as below:

Sequential Access

The system allows only one computer to access the printer at a time. After the computer prints the document, it releases the printer so the next computer gets the turn. For instance, Computer 1 acquired the printer first and once it finished printing, Computer 3 was able to print next. This clearly shows the simulation is keeping the printing jobs queue in proper order.

No Overlapping Jobs

There is no observation that shows two computers were printing at the same time. This has proven that the simulation was successful in preventing race conditions, which usually happens when there is no proper control of who can or cannot access the shared resources.

Proper Release Mechanism

Every thread in the simulation follows the pattern of requesting access, completing its printing job, and then releasing the printer back to the system. There was no sign of any threads keeping the printer locked or failing to release after completing its job.

7.0 Future Work & Conclusion

Challenges

While performing this project, we encountered several simple issues in simulating the access of a shared printer resource by several users using semaphores. The main issue was

simulating valid real-time interaction involving several users and a shared printer resource without producing data inconsistency. While implementing valid synchronization logic in Java employing the Semaphore class, appropriate handling of thread states and potential deadlocks were addressed. It was also difficult to devise a good way of representing the process graphically—in other words, in terms of simulation—especially how to represent all the steps in the queue in the right way visually. Getting the whole system into a readable and understandable format was not easy either because that meant condensing the technicality without compromising precision.

Benefits

Despite the issues, the project was learning-oriented. Building the simulation provided us with an appreciation for semaphores in real application to control shared hardware resources. It solidified our understanding of mutual exclusion, process synchronization, and thread-safe programming. In addition, implementing the print queue logic cultivated our problem-solving skill through learning on how to anticipate and deal with race conditions and deadlocks within a simulated setting.

Experience Gained

This project further enhanced our ability to work as a team. Sharing the workload among design, implementation, testing, and documentation taught us to plan, how to respond when confronted with obstacles, and how to communicate technically through continuous debate of matters and giving feedback to one another. In the future, the functionalities could be the inclusion of support for multiple printers, inclusion of priority queues, and inclusion of a graphical user interface such that the simulation is interactive and not too complicated to use. On the whole, the experience really enhanced our understanding about concurrent programming and highlighted the real-world usage of synchronization methods like semaphores in resolving resource conflicts.

8.0 Acknowledgement

We would like to sincerely thank all of the people that helped us with the preparation of this report, “Synchronization Using Semaphores” in “a Printing Queue System”. Without the guidance, support, and encouragement of many others, this project would not have been able to be finished.

We would first and foremost like to express our gratitude to Dr. Farkhana binti Muchtar, our course lecturer, for his unwavering support and direction during the SECR2043 Operating Systems course. His concise explanations, perceptive criticism, and unwavering dedication to our education were crucial in determining the focus and quality of this project.

We also want to express our sincere gratitude to our team members for their continuous cooperation, commitment, and active participation. Each team member contributed particular skills, and teamwork was essential in navigating challenges and creating an extensive final report and video presentation.

We would also want to express our gratitude to the authors, developers, and contributors of scholarly works and technical materials that improved our understanding of semaphore implementations and operating system synchronization ideas. Our research was based on these resources, which also allowed us to successfully apply theoretical knowledge to a real-world situation.

We also want to express our gratitude to our friends and family for their understanding and moral support during this time. Throughout the process, their support and patience kept us motivated and focused.

Finally, we would like to thank Universiti Teknologi Malaysia (UTM) for giving us access to useful resources like learning platforms, software tools, and libraries as well as a friendly academic environment. The development and successful completion of our project were made possible by these facilities. In closing, we would like to express our sincere gratitude to everyone who helped with this initiative in any manner. Your assistance and participation are much valued and have contributed to the enrichment and fulfillment of this academic journey.

References

Downey, A. (2016). *The Little Book of Semaphores*. <https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf>

VisuAlgo - visualising data structures and algorithms through animation. (2010). Visualgo.net. <https://visualgo.net/en>

IBM Financial Transaction Manager for SWIFT Services. (2025, January 31). Ibm.com. <https://www.ibm.com/docs/en/ftmswsfm324?topic=service-print-queues>

GeeksforGeeks. (2017, September 7). *Semaphores in Process Synchronization*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/semaphores-in-process-synchronization/>

Presentation Video

<https://youtu.be/aplNIJaxvnY>